BiC-MPPI: Bidirectional Clustering MPPI for Long-Horizon Kinodynamic Trajectory Optimization

Minchan Jung, Jeonguk Kim, and Kwang-Ki Kim*

***Abstract*—This paper presents Kinodynamic Bidirectional Clustering Model Predictive Path Integral (BiC-MPPI), a sampling-based trajectory optimizer for long-horizon goal-reaching tasks in cluttered environments. BiC-MPPI retains multiple clustered forward branches from the current state and reverse-time heuristic branches from the goal, associates opposite-direction branch representatives, and uses the resulting connections as references for forward Guide MPPI refinement.**

On 600 modified BARN navigation trials, BiC-MPPI achieved a 97.3% success rate, compared with 47.7–54.2% for the evaluated unidirectional baselines. On 240 simulated 6-DOF manipulator tasks, the best BiC-MPPI configuration achieved a 41.7% success rate, compared with 10.0–22.5% for the evaluated baselines. BiC-MPPI evaluates additional backward and guide rollouts. We therefore report the runtime of each algorithmic stage separately.

Project page: `https://github.com/i-ASL/BiC-MPPI`

I. Introduction

Long-horizon trajectory optimization in cluttered environments is challenging for robotic systems with nonlinear dynamics, input constraints, and collision-avoidance requirements. Sampling-based model predictive control methods, such as Model Predictive Path Integral (MPPI) control, are attractive because they evaluate nonlinear dynamics and non-convex costs through independent rollouts that are well suited to GPU parallelization [1], [2]. However, standard MPPI can struggle in long-horizon goal-reaching tasks: terminal costs may be weakly propagated through finite-horizon samples, and cost-weighted averaging may blend separated homotopy modes into an inferior control sequence.

Classical graph and sampling-based planners provide strong exploration mechanisms [6], [7], but their geometric solutions often require additional smoothing or tracking to satisfy differential constraints. Trajectory-optimization methods instead evaluate candidate controls directly through a system model and have been applied to real-time robotic systems [8]–[10]. Within this class, gradient-based methods such as Differential Dynamic Programming (DDP) and its constrained variants solve local approximations through iterative forward–backward sweeps [11]–[14], whereas MPPI provides a sampling-based alternative through parallel rollout evaluation. Table I summarizes representative planner classes and their practical trade-offs. We next review related kinodynamic planners and MPPI variants.

The authors are with the Department of Electrical and Computer Engineering at Inha University, Republic of Korea. This research was supported by National Research Foundation of Korea (NRF) (RS-2025-00560008, RS-2025-25443510). *Corresponding author (`kwangki.kim@inha.ac.kr`).

Recent kinodynamic planners improve long-range exploration through geometric guidance, problem-conditioned motion primitives, bidirectional informed search, and massively parallel tree expansion [15]–[19]. Although these methods provide effective mechanisms for global exploration, they generally rely on explicit search trees, graphs, learned primitive sets, or local connection procedures that are structurally distinct from rollout-based stochastic trajectory optimization.

Within the MPPI framework, previous studies have improved multimodal rollout handling through clustering [20], sampling feasibility through normal–log-normal perturbations [21], and local control performance through hybridization with IPDDP or ancillary controllers [22], [23]. Nevertheless, these approaches remain predominantly unidirectional, such that long-range goal-reaching behaviors must still be discovered from forward rollouts initialized at the current state. Clustering can preserve distinct rollout modes, but it does not provide an explicit goal-side search structure. Conversely, directly coupling forward and backward searches introduces local-connection and two-point boundary-value difficulties [24]. This limitation motivates the use of heuristic goal-side branches within a forward-rollout MPPI framework.

This paper proposes Bidirectional Clustering MPPI (BiC-MPPI), a parallel sampling-based kinodynamic trajectory optimizer for long-horizon goal-reaching tasks. BiC-MPPI decomposes the search into forward cost-to-come branch generation, backward heuristic cost-to-go branch generation, branch association, and Guide MPPI refinement. The backward branches are not directly executed and are not claimed to be physically executable reverse trajectories; they are used only as heuristic goal-side structures. Forward dynamic consistency is recovered through Guide MPPI refinement, which rolls out candidate controls from the current state.

The contributions of this paper are threefold:

1) We propose a bidirectional clustering MPPI framework that maintains forward cost-to-come branches from the current state and backward heuristic cost-to-go branches from the goal, injecting explicit goal-side structure into the sampling process.
2) We introduce a branch-association and Guide MPPI refinement procedure that converts connected branch candidates into forward-dynamically consistent control sequences.
3) We validate BiC-MPPI on modified BARN-based 2D navigation and simulated 6-DOF manipulator joint-space planning benchmarks, with runtime decomposition and component ablations.

TABLE I
PLANNER-CLASS COMPARISON

Planner	Kin./dyn. model	Cost optimization	GPU parallel	Completeness / optimality	Pros	Cons
Dijkstra	×	✓	×	Complete / graph-optimal	Simple; deterministic shortest path	Graph-dependent feasibility
A*	×	✓	×	Complete / graph-optimal†	Efficient with good heuristic	Heuristic/resolution dependent
RRT	×	×	×	Prob. complete / not optimal	Fast exploration	Requires smoothing/tracking
Kinodynamic RRT*	✓	✓	×	Prob. complete / asympt. optimal [3]	Handles differential constraints	Requires steering/reachability assumptions
Kinodynamic State Lattice	✓	✓	×	Resolution complete / lattice-optimal [4]	Feasible motion primitives; deterministic search	Discretization/primitive-set dependent
BiC-MPPI	✓	✓	✓	No completeness / no formal optimality‡	Dynamics-aware; nonlinear costs	Sampling-budget dependent

The GPU-parallel column indicates whether rollout-level vectorization is intrinsic to the formulation considered here. †With an admissible heuristic. Lattice optimality is restricted to the discretized state lattice and its motion-primitive set. ‡Existing MPPI optimality analyses apply only under method-specific assumptions [5]; no completeness or optimality guarantee is claimed for BiC-MPPI.

The remainder of this paper is organized as follows. Section II reviews the necessary preliminaries, Section III presents BiC-MPPI, Section IV reports simulation results, and Section V concludes the paper.

II. PRELIMINARIES

A. Trajectory Optimization

For state-space trajectory optimization, we consider the state $x_t \in \mathbb{R}^n$ and the input $u_t \in \mathbb{R}^m$ with the stage cost $\psi : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$ and the terminal state cost $\phi : \mathbb{R}^n \to \mathbb{R}$. The associated optimal control problem (OCP) for trajectory optimization can be written as

$$\min_{u_{0:T-1}} \quad \phi(x_T) + \sum_{t=0}^{T-1} \psi(x_t, u_t) \tag{1a}$$

$$\text{subject to} \quad x_0 = x_{\text{init}}, \tag{1b}$$

$$x_{t+1} = f(x_t, u_t), \quad t = 0, \dots, T-1, \tag{1c}$$

$$g(x_t) \le 0, \quad p(x_t) \in \mathcal{F}, \quad t = 0, \dots, T, \tag{1d}$$

$$h(u_t) \le 0, \quad t = 0, \dots, T-1 \tag{1e}$$

where x_{init} is the initial condition, f is the discrete-time dynamics, and g and h define state and input constraints. The terminal cost ϕ encodes the goal-reaching objective. The map p extracts the position component, and $\mathcal{F}$ denotes the collision-free region.

B. Model Predictive Path Integral

MPPI perturbs a nominal control sequence $\bar{U} = (\bar{u}_0, \dots, \bar{u}_{T-1})$ and evaluates the resulting rollouts from x_{init}. The k-th sampled sequence is $U^k = (u_0^k, \dots, u_{T-1}^k)$, where $u_t^k = \bar{u}_t + \varepsilon_t^k$ and $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u)$. For horizon length H, define $\mathbb{U}_H = \{u_{0:H-1} \in \mathbb{R}^{mH} : h(u_t) \le 0,\ t = 0, \dots, H-1\}$. We project sampled input sequences onto this set as

$$\mathcal{P}_{\mathbb{U}_H}(U) = \arg\min_{U' \in \mathbb{U}_H} \|U' - U\|_2 . \tag{2}$$

For time-decoupled input constraints, $\mathbb{U}_H = \mathcal{U}^H$ with $\mathcal{U} = \{u \in \mathbb{R}^m : h(u) \le 0\}$, so projection is applied element-wise. In the reported experiments, the admissible input sets are convex; otherwise, the weighted update can be reprojected onto $\mathbb{U}_H$ [25]. For a collision-free rollout, the MPPI objective is

$$J(U) = \phi(x_T) + \sum_{t=0}^{T-1} \psi(x_t, u_t). \tag{3}$$

Here, x_t is generated by applying U up to time t. A rollout that violates the state or collision constraints is assigned the finite trajectory cost J_{col}. After evaluating N_{s} sampled trajectories, MPPI forms the update

$$w^k = \exp(-\gamma_u J^k), \ \ \bar{w} = \sum_{k=1}^{N_{\text{s}}} w^k, \ \ U^* = \sum_{k=1}^{N_{\text{s}}} \frac{w^k}{\bar{w}} U^k \tag{4}$$

where $J^k = J(U^k)$, $\gamma_u > 0$ is the inverse temperature, and k indexes samples. The exponential weighting favors lower-cost trajectories. If the sampled distribution contains separated low-cost modes, a single weighted average may lie between the modes and produce an inferior or infeasible trajectory. The following subsection describes rollout clustering, which forms separate MPPI updates within the identified rollout modes. In the implementation, the infinite indicator is replaced by a large finite penalty or a fallback rule to avoid a zero-weight update.

C. Rollout Clustering MPPI

Naive MPPI returns a single weighted control sequence even when the sampled low-cost rollouts form separated modes. Let $X(U) = (x_0, \dots, x_T)$ denote the trajectory generated by U. For example, when left- and right-turning obstacle-avoidance rollouts are averaged together, the resulting control sequence can lie between the two modes and produce

$$X(U^*) \notin \mathcal{X}^{T+1},$$

although each mode contains feasible samples.

Rollout Clustering MPPI mitigates this mode-averaging problem by clustering the evaluated rollouts before applying the MPPI update [20]. Let $\mathcal{V} = \{r : J^r < J_{\text{col}}\}$ be the valid rollout-index set, and let D^r denote the clustering feature of rollout r. A clustering method returns a collection of disjoint rollout-index sets

$$\mathcal{C} = \{C_1, \dots, C_I\}, \qquad C_i \subseteq \mathcal{V}.$$

For each cluster, the representative control sequence is computed using the within-cluster MPPI update

$$U^{C_i} = \mathcal{P}_{\mathbb{U}_T}\left(\sum_{k \in C_i} \frac{w^k}{\sum_{\ell \in C_i} w^\ell} U^k \right). \tag{5}$$

In this work, rollout clustering can be instantiated using either K-Means [26] or DBSCAN [27]. K-Means uses a prescribed number of clusters, which provides a fixed branch count and predictable downstream computation, but requires selecting K and assigns every valid sample to a cluster. DBSCAN determines the cluster count from local feature density and can reject sparse samples as noise, but is sensitive

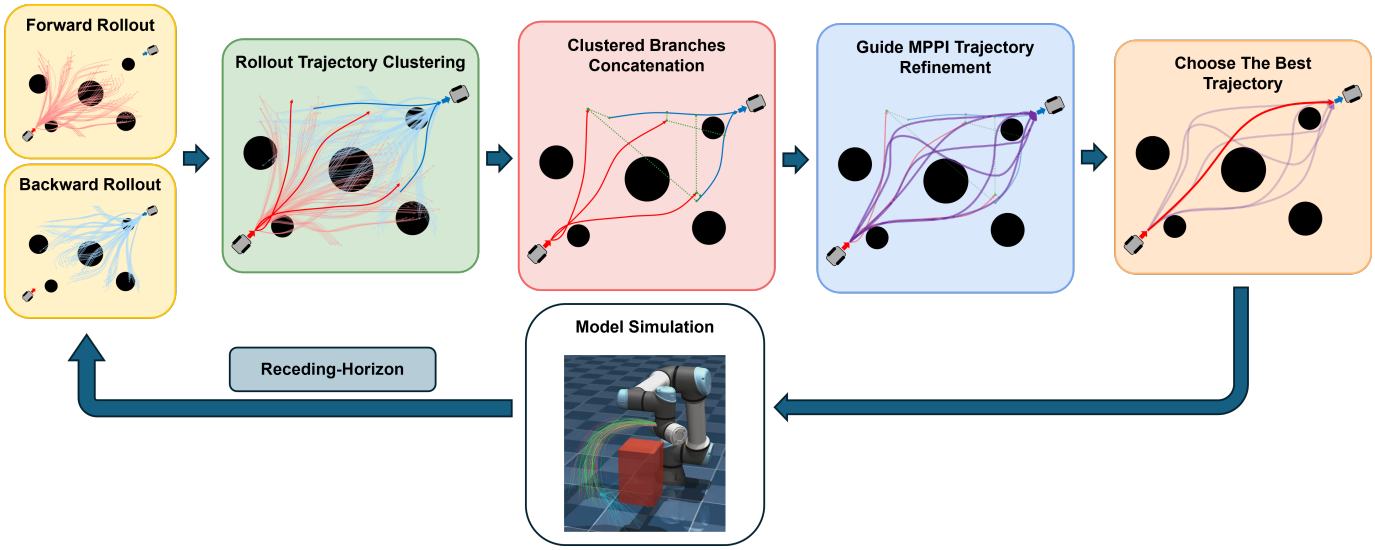

Fig. 1. Overview of the proposed BiC-MPPI framework. The final panel selects the lowest-cost refined candidate and applies its first control in receding-horizon form.

to its density parameters and produces a variable number of branches.

The standard Cluster-MPPI baseline selects the lowest-cost cluster representative,

$$i^\star \in \arg\min_{i\in\{1,\dots,I\}} J(U^{C_i}), \qquad (C^\star, U^{C^\star}) := (C_{i^\star}, U^{C_{i^\star}}). \tag{6}$$

In BiC-MPPI, this immediate single-cluster selection is not performed in the forward and backward stages. Instead, multiple forward and backward cluster representatives are retained for the subsequent branch-association stage.

D. Bidirectional Rollout Kinematics

Forward and backward rollouts use discrete-time maps obtained from numerical integration of the continuous-time model.

a) Forward Rollout Kinematics: For a sampled control sequence $U^k = (u_0^k, \dots, u_{T-1}^k)$, the forward rollout integrates $\dot{x} = F(x,u)$ from x_{init}. The primary 2D implementation uses RK4 with piecewise-constant controls [28]:

$$\begin{aligned} \kappa_1 &= \delta t F(x_t, u_t), \quad \kappa_2 = \delta t F(x_t + \kappa_1/2, u_t), \\ \kappa_3 &= \delta t F(x_t + \kappa_2/2, u_t), \quad \kappa_4 = \delta t F(x_t + \kappa_3, u_t) \\ x_{t+1} &= x_t + (\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4)/6 \end{aligned} \tag{7}$$

where $x_t = x(t\delta t)$, $u_t = u(t\delta t)$, and u_t is held constant on $[t\delta t, (t+1)\delta t)$. This convention uses exactly $u_{0:T-1}$ and avoids an endpoint input u_T.

b) Backward Rollout Kinematics: The backward stage generates goal-side branch candidates by reverse-time integration from $x_T = x_{\text{goal}}$. It uses the same continuous-time dynamics with a negative step and piecewise-constant controls. For $t = T, T-1, \dots, 1$, control u_{t-1} is held constant while integrating one step backward:

$$\begin{aligned} \kappa_1 &= \delta t F(x_t, u_{t-1}), \quad \kappa_2 = \delta t F(x_t - \kappa_1/2, u_{t-1}), \\ \kappa_3 &= \delta t F(x_t - \kappa_2/2, u_{t-1}), \quad \kappa_4 = \delta t F(x_t - \kappa_3, u_{t-1}) \\ x_{t-1} &= x_t - (\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4)/6 \end{aligned} \tag{8}$$

TABLE II
BIC-MPPI NOTATION

Symbol	Description
$U_{\text{f}}, U_{\text{b}}, U_{\text{g}}$	Forward, backward, and guide control sequences
$X_{\text{f}}^i, X_{\text{b}}^j, \hat{X}$	Forward, backward, and concatenated guide trajectories
$\mathcal{B}_{\text{f}}, \mathcal{B}_{\text{b}}$	Retained forward/backward branch representatives
$C_i, \mathcal{C}$	Rollout cluster and cluster collection
$T_{\text{f}}, T_{\text{b}}, T_i$	Forward, backward, and guide horizons
$N_{\text{f}}, N_{\text{b}}, N_{\text{g}}$	Forward, backward, and guide sample counts
$J^k, J_{\text{f}}, J_{\text{b}}, J_{\text{g}}, J_{\text{sel}}$	Rollout, branch, guide, and selection costs
$\mathbb{U}_T$	Length-T admissible control-sequence set

The update in (8) defines the reverse-time sampling guide used by the bidirectional branch generator.

III. BIC-MPPI ALGORITHM

BiC-MPPI decomposes the original trajectory optimization problem into a forward cost-to-come subproblem, a backward cost-to-go subproblem, and a connection-and-refinement subproblem. It consists of three stages: bidirectional clustered branch generation, branch association and connection, and Guide MPPI refinement, as summarized in Fig. 1. In Step 1, the forward branch set is evaluated as a sampling-based cost-to-come search from the current state, and the backward branch set is evaluated as a reverse-time heuristic cost-to-go search from the goal state.

Algorithm 1 gives the complete receding-horizon procedure, in which BiC-MPPI, unlike unidirectional MPPI, retains multiple forward and backward cluster representatives, associates opposite-direction branches, and performs the final selection only after forward Guide MPPI refinement. To support the algorithmic presentation, we use the integer-index shorthand $\mathbb{Z}_{a:b} = \{a, a+1, \dots, b\}$ for integers $a \le b$, place the stage-specific subroutines with their corresponding steps below, and summarize the main symbols of the BiC-MPPI formulation in Table II.

Algorithm 1 BiC-MPPI

Input: initial state x_{init}, goal state x_{goal}
Output: executed control history $\mathcal{U}_{\mathrm{exec}}$
Initialize: initial controls $U_{\mathrm{f}0}, U_{\mathrm{b}0}, \mathcal{U}_{\mathrm{exec}} \leftarrow \emptyset$, $\ell \leftarrow 0$
while goal condition is not satisfied **do**
  Step 1. Bidirectional Clustered Branch Generation
  $\mathcal{B}_{\mathrm{f}} = \{(U_{\mathrm{f}}^i, X_{\mathrm{f}}^i, J_{\mathrm{f}}^i)\}_{i=1}^{I_{\mathrm{f}}} \leftarrow \texttt{ForwardClusterMPPI}(U_{\mathrm{f}0})$ ▷ Alg. 2
  $\mathcal{B}_{\mathrm{b}} = \{(U_{\mathrm{b}}^j, X_{\mathrm{b}}^j, J_{\mathrm{b}}^j)\}_{j=1}^{I_{\mathrm{b}}} \leftarrow \texttt{BackwardClusterMPPI}(U_{\mathrm{b}0})$ ▷ Alg. 2
  Step 2. Branch Association and Connection
  $\{\check{U}^{(i)}, \check{X}^{(i)}, T_i\}_{i=1}^{I_{\mathrm{f}}} \leftarrow \texttt{Connect}(\mathcal{B}_{\mathrm{f}}, \mathcal{B}_{\mathrm{b}})$ ▷ Sec. III-B
  $\mathcal{I}_{\mathrm{conn}} \leftarrow \{i \in \mathbb{Z}_{1:I_{\mathrm{f}}} : T_i > 0\}$
  if $\mathcal{I}_{\mathrm{conn}} = \emptyset$ **then**
    $i^\dagger = \arg\min_{i \in \mathbb{Z}_{1:I_{\mathrm{f}}}} J_{\mathrm{f}}^i, \quad U^{\ell,*} = U_{\mathrm{f}}^{i^\dagger}$
  else
    Step 3. Guide MPPI Refinement
    for each $i \in \mathcal{I}_{\mathrm{conn}}$ **do** // Parallel
      $(U_{\mathrm{g}}^{(i)*}, J_{\mathrm{sel}}^{(i)}) \leftarrow \texttt{GuideMPPI}(\check{U}^{(i)}, \check{X}^{(i)})$ ▷ Alg. 3
    end for
    $i^\star = \arg\min_{i \in \mathcal{I}_{\mathrm{conn}}} J_{\mathrm{sel}}^{(i)}, \quad U^{\ell,*} = U_{\mathrm{g}}^{(i^\star)*}$
  end if
  Receding-horizon execution
  $u_0^{\ell,*} \leftarrow \texttt{First}(U^{\ell,*})$
  Apply $u_0^{\ell,*}$ and observe the next state x_{next}
  $\mathcal{U}_{\mathrm{exec}} \leftarrow \mathcal{U}_{\mathrm{exec}} \oplus u_0^{\ell,*}$
  $x_{\mathrm{init}} \leftarrow x_{\mathrm{next}}$
  $U_{\mathrm{f}0} \leftarrow \texttt{Shift}(U^{\ell,*})$
  $\ell \leftarrow \ell + 1$
end while
return $\mathcal{U}_{\mathrm{exec}}$

A. Step 1: Bidirectional Clustered Branch Generation

Step 1 generates forward and backward clustered branches. Forward branches start from x_{init} and are evaluated as cost-to-come candidates; backward branches start from x_{goal} and are evaluated as heuristic cost-to-go candidates. Unlike the clustering MPPI baseline, BiC-MPPI retains representatives from both directions for later connection. The forward problem is

$$\begin{aligned} \min_{U \in \mathbb{U}_{T_{\mathrm{f}}}} \quad & J_{\mathrm{f}}(U) = \phi_{\mathrm{f}}(x_{T_{\mathrm{f}}}) + \sum_{t=0}^{T_{\mathrm{f}}-1} \psi_{\mathrm{f}}(x_t, u_t) \\ \text{s.t.} \quad & x_{t+1} = f_{\mathrm{f}}(x_t, u_t), \quad t = 0, \dots, T_{\mathrm{f}} - 1, \\ & x_0 = x_{\mathrm{init}}. \end{aligned} \tag{9}$$

Here, f_{f} is obtained from the forward RK4 map in (7). The cost J_{f} scores initial-side branch representatives. Algorithm 2, instantiated with $q = \mathrm{f}$, gives the corresponding forward clustering MPPI subroutine.

The backward stage stores goal-side cluster representatives as heuristic cost-to-go candidates, which are later coupled with the forward cost-to-come branches in the connection stage. The corresponding problem is

$$\begin{aligned} \min_{U \in \mathbb{U}_{T_{\mathrm{b}}}} \quad & J_{\mathrm{b}}(U) = \phi_{\mathrm{b}}(x_0) + \sum_{t=1}^{T_{\mathrm{b}}} \psi_{\mathrm{b}}(x_t, u_{t-1}) \\ \text{s.t.} \quad & x_{t-1} = f_{\mathrm{b}}(x_t, u_{t-1}), \quad t = T_{\mathrm{b}}, \dots, 1, \\ & x_{T_{\mathrm{b}}} = x_{\mathrm{goal}}. \end{aligned} \tag{10}$$

Algorithm 2 Bidirectional cluster MPPI

Input: direction $q \in \{\mathrm{f}, \mathrm{b}\}$, initial controls U_{q0}
Output: directional branches $\mathcal{B}_q = \{(U_q^i, X_q^i, J_q^i)\}_{i=1}^{I_q}$
for $k = 1, \dots, N_q$ **do** // Parallel
  Sample $\varepsilon_{0:T_q-1}^k$ with $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u)$
  $U^k \leftarrow \mathcal{P}_{\mathbb{U}_{T_q}}(U_{q0} + \varepsilon^k)$
  $X^k \leftarrow \texttt{Rollout}_q(x_q, U^k), \quad J^k \leftarrow J_q(U^k)$
  Compute the block-wise control-input clustering feature D^k
end for
$\mathcal{V}_q \leftarrow \{r \in \mathbb{Z}_{1:N_q} : J^r < J_{\mathrm{col}}\}$
if $\mathcal{V}_q = \emptyset$ **then**
  $\mathcal{V}_q \leftarrow \{\arg\min_{r \in \mathbb{Z}_{1:N_q}} J^r\}$
end if
$\mathcal{C}_q = \{C_1, \dots, C_{I_q}\} \leftarrow \texttt{Clustering}(\{D^r\}_{r \in \mathcal{V}_q})$
if $\mathcal{C}_q = \emptyset$ **then**
  $\mathcal{C}_q \leftarrow \{\mathcal{V}_q\}, \quad I_q \leftarrow 1$
end if
for each $C_i \in \mathcal{C}_q$ **do**
  $\bar{J}_i \leftarrow \min_{r \in C_i} J^r, \quad w_i^s \leftarrow \exp[-\gamma_u (J^s - \bar{J}_i)], \; s \in C_i$
  $Z_i \leftarrow \sum_{s \in C_i} w_i^s, \quad U_q^i \leftarrow \mathcal{P}_{\mathbb{U}_{T_q}}\left(\sum_{s \in C_i} \frac{w_i^s}{Z_i} U^s\right)$
  $X_q^i \leftarrow \texttt{Rollout}_q(x_q, U_q^i), \quad J_q^i \leftarrow J_q(U_q^i)$
end for
return $\mathcal{B}_q$

Here, x_{goal} is the terminal boundary condition and f_{b} is obtained from the backward RK4 map in (8). The cost J_{b} is used as a heuristic cost-to-go score for retaining goal-side representatives. Algorithm 2, instantiated with $q = \mathrm{b}$, gives the corresponding backward clustering MPPI subroutine. A colliding forward or backward rollout is assigned J_{col}.

For the common presentation below, let $q \in \{\mathrm{f}, \mathrm{b}\}$ denote the rollout direction. The direction-specific quantities are $(x_q, T_q, N_q, U_{q0}, J_q, \texttt{Rollout}_q)$, where $x_{\mathrm{f}} = x_{\mathrm{init}}$ and $x_{\mathrm{b}} = x_{\mathrm{goal}}$. The forward and backward calls remain independent and use their respective dynamics, costs, horizons, and sample counts.

Once the warm starts are fixed, the forward and backward rollout subproblems are independent and can be evaluated in parallel. The connection stage then couples forward cost-to-come branches with backward cost-to-go branches.

B. Step 2: Branch Association and Connection

Step 2 associates the independently generated forward and backward branches. For each forward branch, the nearest point pair under a connection metric is found, the two branches are trimmed at the selected indices, and the resulting segments are concatenated into a guide candidate for Step 3.

The connection stage operates on the retained branch representatives output by Step 1,

$$\mathcal{B}_{\mathrm{f}} = \{(U_{\mathrm{f}}^i, X_{\mathrm{f}}^i, J_{\mathrm{f}}^i)\}_{i=1}^{I_{\mathrm{f}}}, \qquad \mathcal{B}_{\mathrm{b}} = \{(U_{\mathrm{b}}^j, X_{\mathrm{b}}^j, J_{\mathrm{b}}^j)\}_{j=1}^{I_{\mathrm{b}}}.$$

Here, X_{f}^i and X_{b}^j are representative state trajectories after the within-cluster MPPI update. The connection search uses only the retained representatives $\mathcal{B}_{\mathrm{f}}$ and $\mathcal{B}_{\mathrm{b}}$; the rollout clusters and their assignments remain internal to Algorithm 2. The backward representative is stored in increasing index order with $(X_{\mathrm{b}}^j)_{T_{\mathrm{b}}} = x_{\mathrm{goal}}$.

For each forward branch i, the associated backward branch and cut indices are

$$(j_i, \tau_i^{\mathrm{f}}, \tau_i^{\mathrm{b}}) = \underset{(j,\tau,\tau')}{\arg\min}\, d_{\mathrm{c}}\Big((X_{\mathrm{f}}^i)_{\tau}, (X_{\mathrm{b}}^j)_{\tau'}\Big) \quad \text{s.t.} \quad j \in \mathbb{Z}_{1:I_{\mathrm{b}}}, \quad \tau \in \mathbb{Z}_{0:T_{\mathrm{f}}}, \quad \tau' \in \mathbb{Z}_{0:T_{\mathrm{b}}}. \tag{11}$$

The connection metric d_{c} must be defined in a consistent state space before association is performed. In the released implementation, we use a weighted Euclidean distance between the stored state vectors,

$$d_{\mathrm{c}}(x, x') = \|x - x'\|_W = \sqrt{(x - x')^\top W (x - x')}, \tag{12}$$

where $W \succeq 0$ defines the state-component weighting used for branch association. Depending on the task, this metric can be instantiated as the identity-weight full-state distance, a position-only distance, or a normalized state-space distance by choosing the corresponding weights. More structured choices, such as LQR-based metrics [29], can also be substituted when a task-specific local geometry is available.

Once (11) is solved, τ_i^{f} and τ_i^{b} are the cut indices on the forward branch and selected backward branch, respectively. The two cut states need not be identical; any resulting reference discontinuity is handled by the subsequent Guide MPPI refinement.

The result of this branch association is the concatenation of the state and control input sequences:

$$\begin{aligned} \check{U}^{(i)} &= (U_{\mathrm{f}}^i)_{0:\tau_i^{\mathrm{f}}-1} \oplus (U_{\mathrm{b}}^{j_i})_{\tau_i^{\mathrm{b}}:T_{\mathrm{b}}-1}, \\ \check{X}^{(i)} &= (X_{\mathrm{f}}^i)_{0:\tau_i^{\mathrm{f}}} \oplus (X_{\mathrm{b}}^{j_i})_{\tau_i^{\mathrm{b}}+1:T_{\mathrm{b}}}. \end{aligned} \tag{13}$$

Here, $\oplus$ denotes compatible sequence concatenation and $Y_{a:b} = \emptyset$ when $a > b$.

The concatenated horizon is $T_i = \tau_i^{\mathrm{f}} + (T_{\mathrm{b}} - \tau_i^{\mathrm{b}})$, so $|\check{U}^{(i)}| = T_i$ and $|\check{X}^{(i)}| = T_i + 1$. Candidates with $T_i = 0$ are excluded from $\mathcal{I}_{\mathrm{conn}} = \{i \in \mathbb{Z}_{1:I_{\mathrm{f}}} : T_i > 0\}$; if this set is empty, Algorithm 1 falls back to the lowest-cost forward branch.

C. Step 3: Guide MPPI Refinement

Step 3 refines each connected candidate with Guide MPPI. The concatenated sequence in (13) is used as a reference, and sampled forward rollouts are weighted by a guide cost.

For each connected candidate i, the Guide MPPI refinement problem used in the reported 2D implementation is written as

$$\begin{aligned} \min_{U \in \mathbb{U}_{T_i}} \quad & J_{\mathrm{g,impl}}^{(i)}(U; \check{X}^{(i)}) \\ \text{s.t.} \quad & x_{t+1} = f_{\mathrm{f}}(x_t, u_t), \quad t = 0, 1, \ldots, T_i - 1, \\ & x_0 = x_{\mathrm{init}}. \end{aligned} \tag{14}$$

Here, $U = (u_0, \ldots, u_{T_i-1})$ and $X = (x_0, \ldots, x_{T_i})$ is the corresponding forward rollout. Let $J_{\mathrm{task}}(U)$ denote the task-specific rollout objective. The Guide MPPI objective is

$$J_{\mathrm{g,impl}}^{(i)}(U; \check{X}^{(i)}) = J_{\mathrm{task}}(U) + w_{\mathrm{ref}} \sum_{t=0}^{T_i} \|x_t - \check{x}_t^{(i)}\|_2. \tag{15}$$

A colliding guide rollout is assigned J_{col}. The selected guide cost is used in the MPPI weight update (4), producing a refined input $U_{\mathrm{g}}^{(i)*}$. Final selection removes the reference-deviation term and uses

$$J_{\mathrm{sel}}^{(i)} = J_{\mathrm{task,sel}}(X), \tag{16}$$

with colliding candidates again assigned J_{col}. The concrete task objectives are given in the corresponding experiment subsections. Algorithm 3 summarizes the Guide MPPI refinement used after branch association.

Algorithm 3 Guide MPPI

Input: warm-start input $\check{U}^{(i)}$, reference state trajectory $\check{X}^{(i)}$
Output: guided control input $U_{\mathrm{g}}^{(i)*}$ and selection cost $J_{\mathrm{sel}}^{(i)}$
for $k = 1, \ldots, N_{\mathrm{g}}$ **do** // Parallel
  $\varepsilon^k = \varepsilon^k_{0:T_i-1}$ where $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u),\ t \in \mathbb{Z}_{0:T_i-1}$
  $U^k \leftarrow \mathcal{P}_{\mathbb{U}_{T_i}}(\check{U}^{(i)} + \varepsilon^k)$
  Roll out X^k and evaluate $J^k \leftarrow J_{\mathrm{g,impl}}^{(i)}(U^k; \check{X}^{(i)})$
end for
$\bar{J} \leftarrow \min_{k \in \mathbb{Z}_{1:N_{\mathrm{g}}}} J^k$
for $k = 1, \ldots, N_{\mathrm{g}}$ **do**
  $J^k = J^k - \bar{J},\ w^k = \exp(-\gamma_u J^k)$
end for
$\bar{w} = \sum_{k=1}^{N_{\mathrm{g}}} w^k,\ \widehat{U}_{\mathrm{g}}^{(i)} = \sum_{k=1}^{N_{\mathrm{g}}} (w^k/\bar{w}) U^k$
$U_{\mathrm{g}}^{(i)*} \leftarrow \mathcal{P}_{\mathbb{U}_{T_i}}(\widehat{U}_{\mathrm{g}}^{(i)})$
$J_{\mathrm{sel}}^{(i)} \leftarrow$ the final-selection cost in (16) after rolling out without the guide-deviation term
return $(U_{\mathrm{g}}^{(i)*}, J_{\mathrm{sel}}^{(i)})$

IV. Simulation Experiments

A. Experimental Setup and Implementation Details

All experiments use a C++/CUDA implementation on an Ubuntu 22.04 workstation. Colliding rollouts receive the penalty $J_{\mathrm{col}} = \infty$ before weighting or clustering.

For both clustering variants, each sampled control sequence is represented by its block-wise mean input deviation over $B_{\mathrm{cl}} = 3$ temporal blocks. Define

$$\begin{aligned} \mathcal{I}_q(T) &= \{t \in \mathbb{Z}_{0:T-1} : l_q(T) \le t < u_q(T)\}, \\ l_q(T) &= \left\lfloor \frac{(q-1)T}{B_{\mathrm{cl}}} \right\rfloor, \quad u_q(T) = \left\lfloor \frac{qT}{B_{\mathrm{cl}}} \right\rfloor, \\ & q = 1, \ldots, B_{\mathrm{cl}}, \end{aligned} \tag{17}$$

where $\mathcal{I}_q(T)$ denotes the q-th horizon segment. The forward- and backward-stage features are

$$D_{\mathrm{f}}^k = \begin{bmatrix} \frac{1}{|\mathcal{I}_1(T_{\mathrm{f}})|} \sum_{t \in \mathcal{I}_1(T_{\mathrm{f}})} W_u (u_t^k - u_{\mathrm{f0},t}) \\ \frac{1}{|\mathcal{I}_2(T_{\mathrm{f}})|} \sum_{t \in \mathcal{I}_2(T_{\mathrm{f}})} W_u (u_t^k - u_{\mathrm{f0},t}) \\ \frac{1}{|\mathcal{I}_3(T_{\mathrm{f}})|} \sum_{t \in \mathcal{I}_3(T_{\mathrm{f}})} W_u (u_t^k - u_{\mathrm{f0},t}) \end{bmatrix}, \tag{18}$$

$$D_{\mathrm{b}}^k = \begin{bmatrix} \frac{1}{|\mathcal{I}_1(T_{\mathrm{b}})|} \sum_{t \in \mathcal{I}_1(T_{\mathrm{b}})} W_u (u_t^k - u_{\mathrm{b0},t}) \\ \frac{1}{|\mathcal{I}_2(T_{\mathrm{b}})|} \sum_{t \in \mathcal{I}_2(T_{\mathrm{b}})} W_u (u_t^k - u_{\mathrm{b0},t}) \\ \frac{1}{|\mathcal{I}_3(T_{\mathrm{b}})|} \sum_{t \in \mathcal{I}_3(T_{\mathrm{b}})} W_u (u_t^k - u_{\mathrm{b0},t}) \end{bmatrix}. \tag{19}$$

Here, W_u is an input-normalization matrix and is set to the identity unless otherwise stated. The block-wise feature preserves early, middle, and late control-deviation patterns; $B_{\mathrm{cl}} = 1$ recovers the horizon-averaged feature. Only non-colliding rollouts are clustered. The valid sample-index set is

$$\mathcal{V} = \{r : J^r < J_{\mathrm{col}}\}. \tag{20}$$

The same valid feature set $\{D^r\}_{r\in\mathcal{V}}$ is provided to either K-Means or DBSCAN. For DBSCAN, two features are neighbors when $\mu_{\text{dev}}\|D^r - D^s\|_2 < \epsilon_{\max}$, and a dense cluster requires at least $p_{\min}$ neighboring samples. For K-Means, the valid features are partitioned into $K_{\text{eff}} = \min(K, |\mathcal{V}|)$ clusters.

In the 2D experiments, Algorithm 3 uses (15) for guided sampling and (16) for final candidate selection. Runtime is reported separately because BiC-MPPI adds clustering, connection search, and guide refinement beyond rollout evaluation.

B. Ground-Vehicle Planning

The 2D navigation benchmark uses a differential-drive robot with continuous-time kinematics

$$\dot{x} = v_t\cos(\theta_t), \quad \dot{y} = v_t\sin(\theta_t), \quad \dot{\theta} = w_t,$$

where (x, y) is the planar position, θ is the heading, and $u_t = [v_t, w_t]^\top$. The OCP uses

$$0 \le v_t \le 1.0, \quad |w_t| \le \frac{\pi}{4}, \quad (x_t, y_t) \notin \mathcal{O},$$

where $\mathcal{O}$ is the map obstacle region. The benchmark uses 300 BARN maps [30], each extended from $3\,\text{m} \times 3\,\text{m}$ to $3\,\text{m} \times 5\,\text{m}$ and inflated for footprint-aware collision checking, as shown in Fig. 2. Two initial states, $[0.5, 0.0, \pi/2]^\top$ and $[2.5, 0.0, \pi/2]^\top$, are tested toward $x_{\text{goal}} = [1.5, 5.0, \pi/2]^\top$ on all maps, yielding 600 trials.

For a target state x_{tar}, the experiment rollout objective is

$$J(U; x_{\text{tar}}) = \sum_{t=0}^{T-1} \|x_t - x_{\text{tar}}\|_2 + \|x_T - x_{\text{tar}}\|_2. \tag{21}$$

The forward and backward stages use $x_{\text{tar}} = x_{\text{goal}}$ and $x_{\text{tar}} = x_{\text{init}}$, respectively. For a connected reference, Guide MPPI uses

$$J_{\text{g}}^{(i)} = \|x_{T_i} - x_{\text{goal}}\|_2 + \sum_{t=0}^{T_i} \|x_t - \tilde{x}_t^{(i)}\|_2. \tag{22}$$

A colliding trajectory is assigned $J_{\text{col}} = 10^8$, and final selection evaluates the goal-reaching objective without the reference-deviation term. For branch association, the SE(2) distance weights are set to $w_{xy} = 0.2$ and $w_\theta = 1.0$.

The 2D comparison includes MPPI [1], Log-MPPI [21], and clustering-based variants of Cluster-MPPI [20] and BiC-MPPI. For each clustering-based planner, we evaluate both DBSCAN and K-Means clustering variants, yielding Cluster-MPPI-DBSCAN, Cluster-MPPI-KMeans, BiC-MPPI-DBSCAN, and BiC-MPPI-KMeans under shared Σ_u and matched branch-generation rollout-step budgets. A trial succeeds if, within 200 replanning iterations, $\|p(x_r) - p(x_{\text{goal}})\|_2 < 0.1\,\text{m}$; elapsed computation is reported separately because BiC-MPPI performs additional stages.

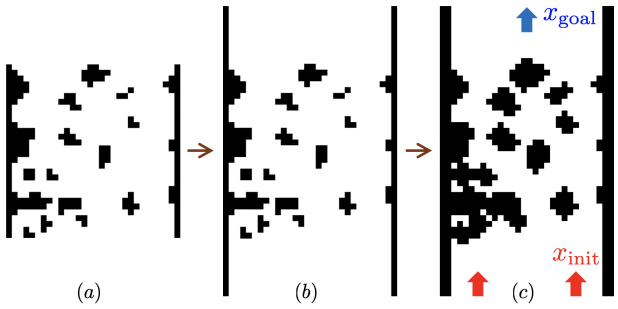

Fig. 2. Modified BARN maps

All variants use shifted warm starts, and trials are initialized independently. Timing is the accumulated wall-clock time averaged over successful trials; failed trials remain included in failure counts and success rates. Table III reports success statistics and runtime decomposition. BiC-MPPI-KMeans succeeds in 97.3% of the 600 trials, whereas BiC-MPPI-DBSCAN succeeds in 89.5%. The two clustering methods exhibit different reliability–computation trade-offs. K-Means provides the highest observed success rate, whereas DBSCAN gives a lower total computation time. The clustering method also affects the downstream connection and Guide MPPI runtimes.

The unidirectional and clustering baselines achieve success rates from 47.7% to 54.2%. The gain comes with additional clustering, branch-connection, and Guide MPPI refinement computation; the stage-wise entries show that clustering and Guide MPPI refinement dominate the accumulated elapsed computation for the bidirectional variants.

TABLE IV
GROUND-VEHICLE EXPERIMENT PARAMETERS

Quantity	Value
State/input dimension	$x \in \mathbb{R}^3$, $u \in \mathbb{R}^2$
Time step	$\Delta t = 0.1$ s
MPPI noise covariance	$\Sigma_u = \text{diag}([0.6, 0.6])$
Inverse temperature	$\gamma_u = 10$
MPPI, Log-MPPI, Cluster-MPPI	$T = 100$, $N = 6000$
BiC-MPPI (ours)	$T_{\text{f}} = 50$, $T_{\text{b}} = 50$, $N_{\text{f}} = N_{\text{b}} = 6000$, $N_{\text{g}} = 3000$
DBSCAN parameters	$\mu_{\text{dev}} = 1.0$, $\epsilon_{\max} = 0.01$, $p_{\min} = 5$
K-Means parameters	$K = 5$, $n_{\text{init}} = 1$, $n_{\text{iter}}^{\max} = 100$
Map resolution	0.1 m/cell

DBSCAN and K-means use the same rollout features; only the clustering rule differs.

TABLE III
GROUND-VEHICLE PLANNING RESULTS OVER 600 TRIALS

Method	Succ. ↑	SR [-] ↑	Fail. ↓	Coll. ↓	Iter. ↓	Total [s] ↓	Roll. [s]	Clust. [s]	Conn. [s]	Guide [s]
MPPI	325	0.542	275	18	144.3	0.330	0.330	0	0	0
Log-MPPI	286	0.477	314	**8**	153.6	**0.291**	0.291	0	0	0
Cluster-MPPI-DBSCAN	313	0.522	287	18	145.2	2.085	0.224	1.862	0	0
Cluster-MPPI-KMeans	287	0.478	313	13	151.3	2.693	0.161	2.532	0	0
BiC-MPPI-DBSCAN (ours)	537	0.895	63	47	94.6	2.540	0.189	2.167	**0.002**	0.182
BiC-MPPI-KMeans (ours)	**584**	**0.973**	**16**	10	**84.1**	3.026	**0.093**	2.356	0.023	0.554

Succ.: successful trials; SR: success rate; Fail.: failed trials; Coll.: collision trials. Times are averaged over successful trials. Stage times may not sum exactly to the total because of rounding and measurement overhead.

C. Manipulator Planning Benchmark

The manipulator benchmark evaluates the same solvers on simulated 6-DOF pose planning using a DH kinematic chain and a decoupled joint-space torque model. Planning is performed in configuration space, while collision checking is evaluated in task space.

The manipulator state and input are

$$x = [q^\top, \dot{q}^\top]^\top \in \mathbb{R}^{12}, \qquad u = \tau \in \mathbb{R}^6,$$

where q, $\dot{q}$, and τ denote joint angles, joint velocities, and torque-like commands. Each joint follows

$$\dot{q}_i = v_i, \qquad \dot{v}_i = \frac{\mathrm{sat}(\tau_i) - d_i v_i - g_i \sin(q_i)}{I_i},$$

where $v_i = \dot{q}_i$ and $\mathrm{sat}(\tau_i)$ clips the command by the torque limit. The coefficients are

$$\begin{aligned} I &= [4.0, 3.5, 2.5, 0.9, 0.6, 0.35], \\ d &= [3.5, 3.2, 2.5, 0.8, 0.5, 0.35], \\ g &= [0.0, 7.0, 4.5, 0.6, 0.3, 0.15]. \end{aligned} \tag{23}$$

Joint-velocity limits are $[2.0, 2.0, 2.0, 2.5, 2.5, 2.5]$ rad/s and torque limits are $[18, 18, 14, 8, 6, 4]$. Forward kinematics use DH parameters $a = [0, -0.427, -0.357, 0, 0, 0]$ m, $d_{\mathrm{DH}} = [0.15, 0, 0, 0.11, 0.09, 0.09]$ m, and $\alpha = [\pi/2, 0, 0, \pi/2, -\pi/2, 0]$ rad. GPU rollouts use explicit Euler integration, while the executed state uses RK4, both with $\Delta t = 0.02$ s.

The manipulator regularization and goal-state costs are

$$\begin{aligned} \ell_{\mathrm{reg}}(x_t, \tau_t) &= 10^{-3}\|\tau_t\|_2^2 + 2 \times 10^{-2}\|\dot{q}_t\|_2^2 \\ &\quad + 20\,\ell_{\mathrm{lim}}(q_t) + \ell_{\mathrm{obs}}(q_t), \end{aligned} \tag{24}$$

$$\begin{aligned} \Phi(x_t, x_g) &= 1500\|q_t - q_g\|_2^2 \\ &\quad + 500\|p_{\mathrm{ee}}(q_t) - p_{\mathrm{ee}}(q_g)\|_2^2 + 100\|\dot{q}_t\|_2^2. \end{aligned} \tag{25}$$

Here, $\ell_{\mathrm{lim}}(q_t)$ penalizes joint configurations outside the central 85% of each admissible joint range, with a quadratic increase toward the joint limits.

The forward rollout objective is

$$\begin{aligned} J_{\mathrm{f}}^{\mathrm{manip}}(U) = &\sum_{t=0}^{T-1} \left[\ell_{\mathrm{reg}}(x_t, \tau_t) + \Phi(x_t, x_g)\right] \\ &+ \Phi(x_T, x_g). \end{aligned} \tag{26}$$

The backward objective has the same form with x_{init} as the target. For a connected reference, the Guide MPPI objective is

$$\begin{aligned} J_{\mathrm{g,manip}}^{(i)} = &\sum_{t=0}^{T_i-1} \left[\ell_{\mathrm{reg}}(x_t, \tau_t) + \Phi(x_t, x_g) \right. \\ &\left. + \|x_t - \breve{x}_t^{(i)}\|_2\right] + \Phi(x_{T_i}, x_g) \\ &+ \|x_{T_i} - \breve{x}_{T_i}^{(i)}\|_2, \end{aligned} \tag{27}$$

and final branch selection uses

$$J_{\mathrm{task,sel}}(X) = \sum_{t=0}^{T_i} \Phi(x_t, x_g). \tag{28}$$

Any hard collision assigns the complete trajectory cost $J_{\mathrm{col}} = 10^8$. Each link is approximated by sampled capsule-like line segments of radius 0.045 m and checked against one AABB obstacle with 0.02 m hard-collision margin.

Fig. 3. Reference postures used for the manipulator pose benchmark.

Tasks are generated from 16 reference postures by evaluating all ordered non-self start–goal pairs, giving $16 \times 15 = 240$ tasks. The postures are converted to zero-velocity states, and one AABB obstacle is placed between each start–goal pair. The obstacle is defined in the world frame as $\mathcal{O}_{\mathrm{box}} = \{p \in \mathbb{R}^3 \mid c_{\mathrm{obs}} - h_{\mathrm{obs}} \le p \le c_{\mathrm{obs}} + h_{\mathrm{obs}}\}$, where c_{obs} is the midpoint of the start and goal end-effector positions and h_{obs} is the obstacle half-size. For all manipulator trials, the obstacle half-size was set to $h_{\mathrm{obs}} = [0.1, 0.1, 0.1]^\top$ m. All solvers share the same task instance and warm-start policy. Fig. 3 shows the reference postures.

A trial is counted as a strict success if

$$\begin{aligned} \|q - q_g\| &< 0.03 \text{ rad}, \\ \|p_{\mathrm{ee}}(q) - p_{\mathrm{ee}}(q_g)\| &< 0.05 \text{ m}, \\ \|\dot{q}\| &< 0.2 \text{ rad/s}, \end{aligned}$$

and the executed trajectory remains collision-free.

TABLE V
MANIPULATOR PLANNING PARAMETERS

Quantity	Value
State/input dimension	$x \in \mathbb{R}^{12}$, $u \in \mathbb{R}^6$
Time step	$\Delta t = 0.02$ s
MPPI noise covariance	$\Sigma_u = \mathrm{diag}([4.5, 4.5, 3.8, 2.2, 1.8, 1.2])$
Inverse temperature	$\gamma_u = 0.0015$
MPPI, Log-MPPI, Cluster-MPPI	$T = 90$, $N = 1024$
BiC-MPPI (ours)	$T_{\mathrm{f}} = 45$, $T_{\mathrm{b}} = 45$, $N_{\mathrm{f}} = N_{\mathrm{b}} = N_{\mathrm{g}} = 1024$
DBSCAN parameters	$\mu_{\mathrm{dev}} = 1.0$, $\epsilon_{\max} = 3.8$, $p_{\min} = 8$
K-Means parameters	$K = 5$, $n_{\mathrm{init}} = 1$, $n_{\mathrm{iter}}^{\max} = 100$

TABLE VI
MANIPULATOR PLANNING RESULTS

Method	Succ.	SR [-]	Strict [-]	EE [-]	CF [-]	Iter. [steps]	Time [s]
MPPI	50	0.208	0.250	0.379	0.708	135.0	3.821
Log-MPPI	53	0.221	0.254	0.375	0.721	134.5	**3.805**
Cluster-MPPI-KM	24	0.100	0.113	0.192	0.721	141.8	5.817
Cluster-MPPI-DB	54	0.225	0.258	0.383	0.721	134.8	5.617
BiC-MPPI-KM (ours)	**100**	**0.417**	**0.471**	**0.546**	**0.738**	92.1	11.599
BiC-MPPI-DB (ours)	96	0.400	0.467	0.529	0.733	**85.5**	4.554

KM: K-Means; DB: DBSCAN; CF: collision-free rate. All rates are reported as fractions over 240 trials. Success uses strict joint, end-effector, velocity, and collision-free conditions. Iteration and wall time are averaged over successful trials only.

Table VI summarizes 240 manipulator tasks. Among the evaluated configurations, BiC-MPPI-KM gives the highest success rate, 41.7%, while BiC-MPPI-DB reaches 40.0%. The corresponding success rates of MPPI, Log-MPPI, Cluster-MPPI-KM, and Cluster-MPPI-DB are 20.8%, 22.1%, 10.0%, and 22.5%, respectively. Between the two BiC-MPPI configurations, DBSCAN reduces the successful-trial wall time from 11.599 s to 4.554 s and the iteration count from 92.1 to 85.5.

Under the tested configurations, both BiC-MPPI variants achieved higher success rates than the forward-only baselines. The component ablation in Table VII separately evaluates the contribution of bidirectional branches. However, BiC-MPPI evaluates additional backward and guide rollouts, so the results should be interpreted as method-configuration comparisons rather than total-computation-matched comparisons.

D. Ablation and Analysis

The ablation studies in this subsection are conducted on the modified BARN-based 2D ground-vehicle navigation benchmark.

a) Component ablation: Table VII isolates the main algorithmic components using the same parameter setting, map set, start states, seed policy, collision checker, and success criterion across all rows.

TABLE VII
COMPONENT ABLATION

Method	SR [-] ↑	Failures ↓	Iter. [steps]	Time [s]
MPPI	0.522	287	104.6	0.244
Cluster-MPPI	<u>0.600</u>	<u>240</u>	101.6	10.949
BiC w/o Cluster	0.927	44	98.0	0.415
BiC w/o Guide	0.882	71	84.0	3.544
Full BiC-MPPI	**0.965**	**21**	93.5	4.238

Times are averaged over successful trials. Bold marks the best overall result; underlining marks the strongest unidirectional baseline.

In Table VII, Cluster-MPPI is the strongest unidirectional baseline, improving success to 60.0% and reducing failures to 240. Full BiC-MPPI gives the best overall reliability, reaching 96.5% success with only 21 failures. Removing Guide MPPI reduces success from 96.5% to 88.2%, with 71 failures and 3.544s average successful-trial time. This variant still connects forward and backward branches, but selects the connected trajectory with the lowest trajectory cost instead of applying Guide MPPI refinement. The BiC w/o Cluster variant reaches 92.7% success with 44 failures and 0.415s average successful-trial time. This variant treats the 8 lowest-cost rollouts in each direction as individual branch representatives instead of applying clustering. The result remains below the full method but above the other ablations, indicating that retained bidirectional branch diversity is important and that explicit clustering further improves reliability.

V. CONCLUSIONS AND FUTURE WORK

This paper presented BiC-MPPI, a bidirectional clustering MPPI framework for long-horizon kinodynamic trajectory optimization in cluttered and multi-modal environments. The method retains multiple forward and backward clustered branch representatives, associates opposite-direction branches through a connection metric, and refines the resulting guide candidates with forward Guide MPPI. The backward branches provide heuristic goal-side references, while the final candidate controls are regenerated through forward rollouts from the current state.

Experiments on modified BARN-based 2D navigation and 6-DOF manipulator planning show that BiC-MPPI improves goal-reaching reliability over MPPI, Log-MPPI, and Cluster-MPPI. Runtime and ablation results indicate that this gain requires additional clustering, branch-connection, and guide-refinement computation, and that both retained branch structure and Guide MPPI refinement contribute to performance. Future work will reduce computational overhead, extend the method to dynamic environments, validate the method in real-time feedback experiments, and develop more dynamics-aware connection metrics.

APPENDIX A
DBSCAN RADIUS SENSITIVITY

TABLE VIII
DBSCAN RADIUS SENSITIVITY

$\epsilon_{\max}$	Rollout [s]	Clustering [s]	Connection [s]	Guide [s]	SR [%] ↑	Time [s] ↓
0.100	0.110	0.016	0.001	0.021	66.3	**0.148**
0.050	0.110	113.639	0.160	0.841	**96.8**	114.750
0.010	0.114	3.620	0.017	0.383	95.3	4.134
0.005	0.115	0.840	0.002	0.142	94.2	1.099
0.001	0.109	1.162	0.001	0.102	93.2	1.375

SR: success rate; times: successful trials.

Table VIII reports the DBSCAN-radius sensitivity sweep used in the 2D experiments. The main experiments use $\epsilon_{\max} = 0.01$ as a reliability–runtime trade-off point. Although $\epsilon_{\max} = 0.050$ gives the highest observed success rate, 96.8%, it requires 114.750s of total elapsed computation.

REFERENCES

[1] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1603–1622, 2018.

[2] M. Kazim, J. Hong, M.-G. Kim, and K.-K. K. Kim, "Recent advances in path integral control for trajectory optimization: An overview in theoretical and algorithmic perspectives," *Annu. Rev. Control*, vol. 57, p. 100931, 2024.

[3] D. J. Webb and J. van den Berg, "Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2013, pp. 5054–5061.

[4] M. Pivtoraiko and A. Kelly, "Differentially constrained motion replanning using state lattices with graduated fidelity," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2008, pp. 2611–2616.

[5] H. Homburger, F. Messerer, M. Diehl, and J. Reuter, "Optimality and suboptimality of MPPI control in stochastic and deterministic settings," *IEEE Control Syst. Lett.*, vol. 9, pp. 763–768, 2025.

[6] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, 2005.

[7] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[8] R. Chai, A. Savvaris, A. Tsourdos, and S. Chai, *Overview of Trajectory Optimization Techniques*. Singapore: Springer, 2020, pp. 7–25.

[9] A. Gasparetto, P. Boscariol, A. Lanzutti, and R. Vidoni, "Path-planning and trajectory-planning algorithms: A general overview," in *Motion and Operation Planning of Robotic Systems*, ser. Mechanisms and Machine Science, G. Carbone and F. Gómez-Bravo, Eds. Cham, Switzerland: Springer, 2015, vol. 29, pp. 3–27.

[10] H.-H. Kwon, H.-S. Shin, Y.-H. Kim, and D.-H. Lee, "Trajectory optimization for impact angle control based on sequential convex programming," *Trans. Korean Inst. Elect. Eng.*, vol. 68, no. 1, pp. 159–166, 2019.

[11] Y. Tassa, N. Mansard, and E. Todorov, "Control-limited differential dynamic programming," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2014, pp. 1168–1175.

[12] Z. Xie, C. K. Liu, and K. Hauser, "Differential dynamic programming with nonlinear constraints," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 695–702.

[13] A. Pavlov, I. Shames, and C. Manzie, "Interior point differential dynamic programming," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 6, pp. 2720–2727, 2021.

[14] T. A. Howell, B. E. Jackson, and Z. Manchester, "ALTRO: A fast solver for constrained trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2019, pp. 7674–7679.

[15] J. Chatrola, A. Ajith, K. Leahy, and C. Chamzas, "Multi-layer motion planning with kinodynamic and spatio-temporal constraints," in *Proceedings of the 28th ACM International Conference on Hybrid Systems: Computation and Control*, 2025, accepted to HSCC 2025; arXiv:2503.07762.

[16] J. Franke, A. Moldagalieva, P. Hanfeld, and W. Hönig, "Accelerating db-a* for kinodynamic motion planning using diffusion," *arXiv preprint arXiv:2503.05539*, 2025.

[17] Y. Wang, B. Mu, S. Shokouhi, and M.-W. Thein, "Optimal kinodynamic motion planning through anytime bidirectional heuristic search with tight termination condition," *arXiv preprint arXiv:2604.11587*, 2026.

[18] N. Perrault, Q. H. Ho, and M. Lahijanian, "Kino-PAX$^+$: Near-optimal massively parallel kinodynamic sampling-based motion planner," *arXiv preprint arXiv:2602.02846*, 2026.

[19] Y. Gao, A. Lu, and Z. Kingston, "Fast asymptotically optimal kinodynamic planning via vectorization," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2026, accepted to IROS 2026; arXiv:2607.03987.

[20] S. Patrick and E. Bakolas, "Path integral control with rollout clustering and dynamic obstacles," in *2024 American Control Conference (ACC)*. IEEE, 2024, pp. 809–814.

[21] I. S. Mohamed, K. Yin, and L. Liu, "Autonomous navigation of AGVs in unknown cluttered environments: Log-MPPI control strategy," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 10 240–10 247, 2022.

[22] M.-G. Kim, M. Jung, J. Hong, and K.-K. K. Kim, "MPPI-IPDDP: A hybrid method of collision-free smooth trajectory generation for autonomous robots," *IEEE Trans. Ind. Informat.*, vol. 21, no. 7, pp. 5037–5046, 2025.

[23] E. Trevisan and J. Alonso-Mora, "Biased-MPPI: Informing sampling-based model predictive control by fusing ancillary controllers," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5871–5878, 2024.

[24] S. Nayak and M. W. Otte, "Bidirectional sampling-based motion planning without two-point boundary value solution," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3636–3654, 2022.

[25] H. H. Bauschke and J. M. Borwein, "On projection algorithms for solving convex feasibility problems," *SIAM Rev.*, vol. 38, no. 3, pp. 367–426, 1996.

[26] S. Lloyd, "Least squares quantization in pcm," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[27] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining (KDD)*, Portland, OR, USA, 1996, pp. 226–231.

[28] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Hoboken, NJ, USA: Wiley, 2016.

[29] R. Tedrake, "LQR-trees: Feedback motion planning on sparse randomized trees," in *Proc. Robot.: Sci. Syst. (RSS)*, Seattle, WA, USA, 2009.

[30] D. Perille, A. Truong, X. Xiao, and P. Stone, "Benchmarking metric ground navigation," in *Proc. IEEE Int. Symp. Safety, Security, Rescue Robot. (SSRR)*, 2020, pp. 116–121.